



**ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR**

Membre de
HONORIS UNITED UNIVERSITIES

EMSI Casablanca

Filière Informatique — 4^e IAD

Année universitaire 2025–2026

PROJET DE FIN DE MODULE

Deep Learning

*Conception, implémentation, comparaison et analyse critique
de modèles de Deep Learning pour données tabulaires, images et séquences*

Réalisé par : Safaa Hassani

Encadrant : Mr. Batal

Framework : PyTorch 2.10

Environnement : Google Colab — CPU

Résumé

Ce rapport présente les travaux réalisés dans le cadre du projet de fin de module Deep Learning à l'EMSI Casablanca (filière 4 IAD, 2025–2026). L'objectif central est d'étudier, d'implémenter et de comparer les trois grandes familles d'architectures neuronales profondes :

- **Partie I — Perceptron Multicouche (MLP)** appliqué à la détection du cancer du sein (dataset *Breast Cancer Wisconsin*) : classification binaire malin / bénin à partir de mesures cellulaires, avec normalisation, Dropout, Weight Decay, comparaison des initialisations, Early Stopping et comparaison aux modèles classiques.
- **Partie II — Réseau de Neurones Convolutionnel (CNN, architecture LeNet)** pour la classification d'images couleur *CIFAR-10* (10 catégories), avec implémentation manuelle des opérations convolutionnelles et comparaison CNN vs MLP.
- **Partie III — Modèles récurrents (RNN, LSTM, GRU) et système Seq2Seq** encodeur–décodeur pour la traduction automatique anglais → espagnol sur un corpus réel (*Tatoeba*), avec gradient clipping, teacher forcing, beam search et évaluation BLEU.

Tous les modèles ont été développés sous PyTorch et entraînés sur Google Colab (CPU).

Mots-clés : Deep Learning, MLP, CNN, LeNet, RNN, LSTM, GRU, Seq2Seq, Breast Cancer Wisconsin, CIFAR-10, Tatoeba, PyTorch, Régularisation, Traduction automatique.

Table des matières

Résumé	1
Introduction Générale	5
1 Partie I — Perceptron Multicouche (MLP)	6
1.1 Présentation du problème.....	6
1.2 Dataset : Breast Cancer Wisconsin.....	6
1.2.1 Répartition des classes.....	6
1.3 Analyse Exploratoire des Données.....	6
1.4 Prétraitement et Normalisation.....	7
1.4.1 Pourquoi normaliser ?.....	7
1.4.2 Prévention du data leakage	7
1.4.3 Séparation Train / Validation / Test.....	7
1.5 Architecture du MLP.....	7
1.5.1 Qu'est-ce qu'un MLP ?.....	7
1.5.2 Deux implémentations équivalentes	7
1.5.3 Architecture finale : $30 \rightarrow 64 \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0,3) \rightarrow 2$	7
1.6 Étude des Paramètres du Réseau.....	8
1.7 Comparaison des Stratégies d'Initialisation.....	8
1.8 Régularisation : Dropout et Weight Decay.....	8
1.9 Early Stopping.....	9
1.10 Entraînement et Hyperparamètres.....	9
1.11 Résultats et Analyse.....	9
1.11.1 Évolution pendant l'entraînement	9
1.11.2 Résultats sur le jeu de test.....	10
1.11.3 Comparaison avec des modèles classiques.....	10
1.11.4 Analyse et discussion.....	10
2 Partie II — Réseaux de Neurones Convolutionnels (CNN)	11
2.1 Présentation du problème.....	11
2.2 Dataset : CIFAR-10.....	11
2.3 Fondements des Opérations Convolutionnelles	11
2.3.1 La corrélation croisée 2D : l'opération de base.....	11
2.3.2 Padding, Stride et Taille de sortie.....	11
2.3.3 Max Pooling et Average Pooling.....	12
2.3.4 Convolution 1×1	12
2.4 Architecture LeNet (adaptée à CIFAR-10).....	12
2.4.1 Propagation des dimensions à travers le réseau.....	12
2.4.2 Nombre de paramètres	12
2.5 Entraînement.....	13
2.6 Résultats et Analyse.....	13
2.6.1 Évolution des performances	13
2.6.2 Performances par classe.....	14
2.6.3 Visualisation des feature maps.....	14
2.6.4 Comparaison CNN vs MLP.....	14

2.6.5	Analyse et discussion.....	14
3	Partie III — RNN, LSTM, GRU et Seq2Seq	15
3.1	Présentation du problème.....	15
3.2	Corpus anglais–espagnol (réel — Tatoeba).....	15
3.2.1	Source et nature des données.....	15
3.2.2	Prétraitement	15
3.3	Modèle de Langage et Perplexité	16
3.4	Architectures Récurrentes.....	16
3.4.1	RNN simple.....	16
3.4.2	LSTM (Long Short-Term Memory)	16
3.4.3	GRU (Gated Recurrent Unit).....	16
3.5	BPTT et Gradient Clipping.....	17
3.6	Comparaison RNN / LSTM / GRU	17
3.7	Système Seq2Seq Encodeur–Décodeur.....	17
3.7.1	Teacher Forcing.....	17
3.8	Stratégies de Décodage.....	18
3.8.1	Décodage glouton	18
3.8.2	Beam Search ($k = 3$).....	18
3.9	Évaluation : Score BLEU.....	18
3.10	Résultats et Analyse.....	18
3.10.1	Analyse et discussion des résultats.....	18
4	Synthèse et Comparaison des Trois Approches	20
4.1	Explication des trois parties.....	20
4.1.1	Partie I — MLP et données tabulaires	20
4.1.2	Partie II — CNN et images.....	20
4.1.3	Partie III — RNN / Seq2Seq et séquences.....	20
4.2	Comparaison transversale	20
4.3	Lecture comparative	21
	Conclusion Générale	22

Liste des tableaux

Table 1 — Vue d'ensemble des architectures étudiées.....	5
Table 1.1 — Caractéristiques du dataset Breast Cancer Wisconsin	6
Table 1.2 — Répartition des classes — Breast Cancer Wisconsin.....	6
Table 1.3 — Rôle et taille de chaque ensemble de données	7
Table 1.4 — Détail des couches et paramètres du MLP	7
Table 1.5 — Comparaison des méthodes d'initialisation testées.....	8
Table 1.6 — Hyperparamètres d'entraînement — MLP Breast Cancer.....	9
Table 1.7 — Progression de l'entraînement — MLP Breast Cancer.....	9
Table 1.8 — Performances finales — MLP Breast Cancer (jeu de test).....	10
Table 1.9 — MLP vs modèles classiques — Breast Cancer Wisconsin.....	10
Table 2.1 — Caractéristiques du dataset CIFAR-10.....	11
Table 2.2 — Paramètres d'une couche convolutionnelle et leur effet	11
Table 2.3 — Évolution des dimensions (entrée : $3 \times 32 \times 32$)	12
Table 2.4 — Décompte des paramètres de LeNet (total : 83 126)	12
Table 2.5 — Hyperparamètres d'entraînement — LeNet CIFAR-10.....	13
Table 2.6 — Progression de la perte et de l'accuracy — LeNet sur CIFAR-10	13
Table 2.7 — Précision, rappel et F1 par classe (jeu de test).....	14
Table 2.8 — Comparaison MLP vs LeNet (CNN) — CIFAR-10, 8 époques.....	14
Table 3.1 — Exemples de paires du corpus réel (après normalisation)	15
Table 3.2 — Statistiques du corpus	15
Table 3.3 — Les trois portes du LSTM et leur rôle.....	16
Table 3.5 — Résultats comparatifs — modèle de langage (corpus réel)	17
Table 3.6 — Hyperparamètres du système Seq2Seq.....	17
Table 3.7 — Performances du système Seq2Seq (corpus réel EN → ES)	18
Table 3.8 — Exemples de traductions (décodage glouton).....	18
Table 4.1 — Comparaison transversale des trois approches.....	20
Table 4.2 — Performances obtenues sur les trois parties du projet	22

Introduction Générale

Le Deep Learning (apprentissage profond) est une branche de l'intelligence artificielle qui utilise des réseaux de neurones à plusieurs couches pour apprendre automatiquement des représentations utiles à partir de données brutes. Contrairement au Machine Learning classique où l'ingénieur choisit manuellement les caractéristiques, le réseau les apprend lui-même.

Aujourd'hui, le Deep Learning est au cœur des avancées en reconnaissance d'images, traduction automatique, détection médicale et génération de texte.

Table 1 — Vue d'ensemble des architectures étudiées

Partie	Architecture	Dataset	Type de données
I	MLP (Perceptron Multicouche)	Breast Cancer Wisconsin	Tabulaires
II	CNN (LeNet)	CIFAR-10	Images
III	RNN / LSTM / GRU / Seq2Seq	Corpus EN → ES (Tatoeba)	Séquences textuelles

Chaque architecture est naturellement adaptée à un type de données : le MLP traite les données tabulaires, le CNN exploite la structure spatiale des images, et les modèles récurrents capturent les dépendances dans les séquences. Les expériences ont été réalisées sur Google Colab avec PyTorch ; les trois parties ont été exécutées sur CPU.

1. Partie I — Perceptron Multicouche (MLP)

1.1 Présentation du problème

La détection du cancer du sein consiste à classer une tumeur comme **bénigne** (sans danger) ou **maligne** (cancéreuse) à partir de mesures réalisées sur des images de cellules. C'est un problème de classification binaire sur des données médicales tabulaires.

Ce problème est important : une erreur de diagnostic peut avoir des conséquences graves. Détecter un cancer malin comme bénin retarde le traitement ; l'inverse génère anxiété et examens inutiles. Le choix des métriques d'évaluation doit en tenir compte.

1.2 Dataset : Breast Cancer Wisconsin

Table 1.1 — Caractéristiques du dataset Breast Cancer Wisconsin

Propriété	Valeur
Nombre d'observations	569 patientes
Nombre de variables	30 mesures numériques
Valeurs manquantes	Aucune
Tâche	Classification binaire (Malin / Bénin)
Source	UCI / scikit-learn (load_breast_cancer)

Les 30 variables décrivent des propriétés géométriques des noyaux cellulaires (rayon, texture, périmètre, aire, lissé, compacité, concavité, symétrie, dimension fractale), chacune calculée de trois façons (moyenne, écart-type, valeur maximale), soit $10 \times 3 = 30$ variables.

1.2.1 Répartition des classes

Table 1.2 — Répartition des classes — Breast Cancer Wisconsin

Classe	Signification	Effectif	Proportion
0	Malin (Malignant) — tumeur cancéreuse	212	37,3 %
1	Bénin (Benign) — tumeur non cancéreuse	357	62,7 %
Total	—	569	100 %

Le jeu est légèrement déséquilibré (37 % / 63 %), représentatif d'une situation clinique réelle. La séparation des ensembles est **stratifiée** pour préserver ces proportions (benign $\approx 62,7$ % dans chaque sous-ensemble).

1.3 Analyse Exploratoire des Données

- **Plages de valeurs très différentes** : certaines variables valent autour de 0,01 (lissé) tan-dis que d'autres dépassent 2 000 (aire). Sans normalisation, les variables à grande échelle domineraient l'apprentissage.
- **Corrélations entre variables** : rayon, périmètre et aire sont fortement corrélés (propriétés géométriques liées) ; le MLP gère cette redondance.
- **Pouvoir discriminant** : rayon, périmètre et concavité séparent bien Bénin et Malin, ce qui rend la tâche réalisable avec un modèle simple.

1.4 Prétraitement et Normalisation

1.4.1 Pourquoi normaliser ?

Les 30 variables ayant des échelles très différentes, on applique une normalisation Z-score : $x_{\text{norm}} = (x - \text{moyenne}) / \text{écart-type}$.

1.4.2 Prévention du data leakage

Qu'est-ce que le data leakage ?

Calculer la moyenne et l'écart-type sur **toutes** les données (test compris) donnerait au modèle un accès indirect à des statistiques du test, d'où des performances artificiellement gonflées. La bonne pratique : ajuster le StandardScaler **uniquement sur le train**, puis l'appliquer à la validation et au test.

En pratique, on sépare d'abord le test de façon stratifiée, puis la validation à partir du reste ; le StandardScaler est ensuite ajusté sur le **seul** ensemble d'entraînement, avant d'être appliqué tel quel à la validation et au test.

1.4.3 Séparation Train / Validation / Test

Table 1.3 — Rôle et taille de chaque ensemble de données

Ensemble	Exemples	Proportion	Rôle
Entraînement	397	70 %	Ajuster les poids du réseau
Validation	86	15 %	Surveiller le surapprentissage, choisir le modèle
Test	86	15 %	Évaluation finale et honnête du modèle
Total	569	100 %	—

1.5 Architecture du MLP

1.5.1 Qu'est-ce qu'un MLP ?

Un Perceptron Multicouche est un réseau entièrement connecté : le signal se propage de l'entrée vers la sortie, puis les erreurs se rétropropagent pour ajuster les poids. L'activation **ReLU** introduit la non-linéarité ; sans elle, plusieurs couches linéaires équivalaient à une seule.

1.5.2 Deux implémentations équivalentes

Le réseau peut être défini de deux manières strictement équivalentes : avec `nn.Sequential`, qui empile les couches dans l'ordre (Linear 30 → 64, ReLU, Dropout 0,3, Linear 64 → 2), ou via une classe héritant de `nn.Module` dont la méthode `forward` décrit explicitement le passage avant. Les deux produisent le même réseau et le même nombre de paramètres (2 114).

1.5.3 Architecture finale : 30 → 64 → ReLU → Dropout(0,3) → 2

Table 1.4 — Détail des couches et paramètres du MLP

#	Couche	Entrée	Sortie	Paramètres	Rôle
1	Linear	30	64	$30 \times 64 + 64 = 1\,984$	Transformation linéaire
–	ReLU	64	64	0	Non-linéarité
–	Dropout	64	64	0	Régularisation (30 %)
2	Linear	64	2	$64 \times 2 + 2 = 130$	Score par classe
	Total			2 114	—

1.6 Étude des Paramètres du Réseau

L’inspection via `named_parameters()` confirme la répartition : la couche cachée porte $64 \times 30 = 1\,920$ poids (plus 64 biais) et la couche de sortie $2 \times 64 = 128$ poids (plus 2 biais), soit **2 114** paramètres apprenables. Le `state_dict` contient tous les poids et biais ; c’est lui qu’on sauvegarde et recharge pour réutiliser un modèle entraîné.

1.7 Comparaison des Stratégies d’Initialisation

Pourquoi l’initialisation est-elle importante ?

Un mauvais choix de poids initiaux peut faire **s’évanouir** le gradient (signal trop faible, le réseau n’apprend plus) ou le faire **exploser** (entraînement qui diverge). Trois stratégies ont été comparées sur la validation.

Table 1.5 — Comparaison des méthodes d’initialisation testées

Méthode	Principe	Val. accuracy
Gaussienne	Loi normale $N(0, 0,01)$	1,0000 (retenue)
Constante	Tous les poids fixés à 1,0 (dégénéré : neurones identiques)	0,9651
Xavier uniform	Intervalle calculé selon entrées/sorties	0,9884

Sur ce découpage, c’est l’initialisation **gaussienne** qui a obtenu la meilleure accuracy de validation (1,0000) et a donc été retenue. L’initialisation constante, bien que correcte numériquement ici, est à éviter : tous les neurones cachés calculant la même chose, la symétrie n’est pas brisée.

1.8 Régularisation : Dropout et Weight Decay

Deux techniques limitent le surapprentissage :

- **Dropout (0,3)** : désactive aléatoirement 30 % des neurones à chaque pas d’entraînement, forçant le réseau à ne pas dépendre de neurones particuliers (désactivé automatiquement à l’évaluation).
- **Weight Decay (L2, 10^{-4})** : pénalise les poids trop grands, intégré directement dans l’optimiseur Adam.

1.9 Early Stopping

Principe de l'Early Stopping

On surveille la perte de validation après chaque époque. Si elle ne s'améliore plus pendant patience = 20 époques consécutives, on arrête et on conserve (via deepcopy) les poids correspondant à la **meilleure** validation, pas à la dernière époque.

1.10 Entraînement et Hyperparamètres

Table 1.6 — Hyperparamètres d'entraînement — MLP Breast Cancer

Hyperparamètre	Valeur	Justification
Optimiseur	Adam	Adaptatif, robuste
Taux d'apprentissage	0,001	Valeur standard
Weight Decay	10^{-4}	Régularisation L2
Dropout	0,3	30 % des neurones désactivés
Fonction de perte	CrossEntropyLoss	Adaptée à la classification
Initialisation	Gaussienne	Meilleure accuracy de validation
Époques (max)	200	Avec early stopping (patience 20)
Device	CPU	Exécution sur Google Colab

1.11 Résultats et Analyse

1.11.1 Évolution pendant l'entraînement

Table 1.7 — Progression de l'entraînement — MLP Breast Cancer

Époque	Train Loss	Val Loss	Val Accuracy
0	0,6934	0,6900	86,0 %
25	0,4331	0,4022	95,3 %
50	0,1791	0,1343	97,7 %
75	0,1145	0,0686	98,8 %
100	0,0893	0,0446	100,0 %
125	0,0730	0,0326	100,0 %
150	0,0659	0,0255	100,0 %
175	0,0570	0,0216	100,0 %

La perte de validation décroît régulièrement et l'accuracy de validation atteint 100 % : le modèle apprend une frontière nette dans l'espace normalisé.

1.11.2 Résultats sur le jeu de test

Table 1.8 — Performances finales — MLP Breast Cancer (jeu de test)

Métrique	Valeur	Interprétation
Accuracy	94,19 %	Très bonne classification globale
Precision	93,31 %	Peu de fausses alarmes
Recall	94,73 %	Peu de cas positifs manqués
F1-Score	93,89 %	Bon équilibre précision/rappel

En contexte médical, le **rappel sur la classe maligne** est la métrique la plus critique : manquer un cancer (faux négatif) est plus grave qu’une fausse alarme. La matrice de confusion permet de vérifier la répartition exacte des erreurs.

1.11.3 Comparaison avec des modèles classiques

Table 1.9 — MLP vs modèles classiques — Breast Cancer Wisconsin

Modèle	Description	Accuracy	F1-Score
Régression Logistique	Modèle linéaire, interprétable	96,51 %	0,963
MLP (notre modèle)	Réseau neuronal régularisé	94,19 %	0,939
Forêt Aléatoire	Ensemble de 300 arbres	91,86 %	0,914

1.11.4 Analyse et discussion

- Le MLP atteint **94,2 % d’accuracy** et un **F1 de 0,94** sur le test, avec un rappel élevé (0,947) — un comportement adapté à un contexte de dépistage.
- Sur ce petit dataset, la **Régression Logistique** obtient le meilleur score (96,5 %) : la frontière de décision est quasi-linéaire dans l’espace normalisé, ce qui avantage un modèle simple et bien régularisé.
- Le MLP devance la **Forêt Aléatoire** (91,9 %) et reste compétitif avec la régression logistique, ce qui valide l’architecture choisie.
- L’écart faible entre pertes d’entraînement et de validation montre que le **Dropout** et le **Weight Decay** limitent le surapprentissage.
- L’**Early Stopping** et la conservation du meilleur modèle garantissent d’évaluer le réseau à son meilleur point de généralisation.

2. Partie II — Réseaux de Neurones Convolutionnels (CNN)

2.1 Présentation du problème

La tâche consiste à classer une image couleur 32×32 dans l'une de 10 catégories (avion, voiture, oiseau, chat, cerf, chien, grenouille, cheval, bateau, camion). C'est une classification à 10 classes sur des images naturelles.

Ce problème illustre la limite du MLP sur les images : aplatir une image $3 \times 32 \times 32$ en un vecteur ferait perdre toute l'information spatiale (quels pixels sont voisins). Le CNN est conçu pour exploiter cette structure.

2.2 Dataset : CIFAR-10

Table 2.1 — Caractéristiques du dataset CIFAR-10

Propriété	Valeur
Images d'entraînement	50 000
Images de test	10 000
Résolution	32×32 pixels
Type	Couleur (3 canaux RGB)
Nombre de classes	10

Prétraitement : chaque canal est normalisé avec une moyenne et un écart-type de 0,5 (valeurs ramenées dans $[-1, 1]$). Les images sont chargées en mini-batches de 64.

2.3 Fondements des Opérations Convolutionnelles

2.3.1 La corrélation croisée 2D : l'opération de base

Comment fonctionne une convolution ?

On fait glisser un filtre (ou noyau) sur toute l'image ; à chaque position, on calcule la somme des produits entre le filtre et la région recouverte. Le résultat est une **carte de caractéristiques** (feature map). En empilant plusieurs filtres, le CNN construit une représentation riche de l'image.

L'opération a été implémentée manuellement en NumPy (double boucle sur les positions, somme des produits filtre $\times$ région), puis comparée à PyTorch : les deux donnent un résultat strictement identique (par exemple $[[19, 25], [37, 43]]$ sur un cas test 3×3).

2.3.2 Padding, Stride et Taille de sortie

Table 2.2 — Paramètres d'une couche convolutionnelle et leur effet

Paramètre	Définition	Effet
Stride (s)	Pas de déplacement du filtre	Un stride de 2 divise la taille de sortie par 2
Padding (p)	Ajout de zéros autour de l'image	Contrôle la taille de sortie ; avec $p = (k - 1)/2$, la taille est conservée

La taille de la carte de sortie se calcule : $H_{\text{sortie}} = \lfloor (H_{\text{entrée}} + 2p - k)/s \rfloor + 1$.

2.3.3 Max Pooling et Average Pooling

Après une convolution, une opération de pooling réduit la taille spatiale :

- **Max Pooling** : retient la valeur maximale de chaque région ; robuste aux petits décalages. C’est l’opération utilisée dans ce réseau. L’implémentation manuelle a été validée contre PyTorch (exemple 2×2 : $[[5, 7], [13, 15]]$, identique).
- **Average Pooling** : calcule la moyenne de chaque région, information plus lissée.

2.3.4 Convolution 1×1

Une convolution avec un filtre 1×1 ne regarde qu’un seul pixel à la fois. Son rôle est de combiner les canaux entre eux sans toucher aux dimensions spatiales : elle permet de réduire ou d’augmenter le nombre de canaux.

2.4 Architecture LeNet (adaptée à CIFAR-10)

L’architecture LeNet, inspirée du modèle de Yann LeCun (1998), alterne convolution et pooling, puis termine par des couches entièrement connectées. Ici elle est adaptée aux images couleur (3 canaux d’entrée), utilise des activations **ReLU** et du **Max Pooling** (plus modernes que les Sigmoid / Average Pooling d’origine). Concrètement, l’enchaînement est : Conv(3 $\rightarrow$ 6, noyau 5×5 , padding 2) suivie d’un MaxPool, puis Conv(6 $\rightarrow$ 16, noyau 5×5) suivie d’un MaxPool, et enfin trois couches entièrement connectées (576 $\rightarrow$ 120 $\rightarrow$ 84 $\rightarrow$ 10) ; toutes les activations sont des ReLU. Le réseau compte 83 126 paramètres.

2.4.1 Propagation des dimensions à travers le réseau

Table 2.3 — Évolution des dimensions (entrée : $3 \times 32 \times 32$)

Couche	Opération	Sortie	Taille
Entrée	Image originale	$3 \times 32 \times 32$	3 072
Conv1	6 filtres 5×5 , $p = 2$	$6 \times 32 \times 32$	6 144
MaxPool1	2×2 , stride 2	$6 \times 16 \times 16$	1 536
Conv2	16 filtres 5×5	$16 \times 12 \times 12$	2 304
MaxPool2	2×2 , stride 2	$16 \times 6 \times 6$	576
Flatten	Aplatissage	576	576
FC1	Linéaire	120	120
FC2	Linéaire	84	84
FC3	Linéaire (sortie)	10	10

2.4.2 Nombre de paramètres

Table 2.4 — Décompte des paramètres de LeNet (total : 83 126)

Couche	Calcul	Paramètres
Conv1	$6 \times (3 \times 25 + 1)$	456
Conv2	$16 \times (6 \times 25 + 1)$	2 416
FC1	$576 \times 120 + 120$	69 240
FC2	$120 \times 84 + 84$	10 164
FC3	$84 \times 10 + 10$	850
Total	—	83 126

2.5 Entraînement

Table 2.5 — Hyperparamètres d’entraînement — LeNet CIFAR-10

Hyperparamètre	Valeur	Justification
Nombre d’époques	8	Convergence raisonnable sur CPU
Taille de batch	64	Bon équilibre vitesse/stabilité
Optimiseur	Adam	Standard et efficace
Taux d’apprentissage	0,001	Valeur classique
Fonction de perte	CrossEntropyLoss	Adaptée aux 10 classes
Activation	ReLU	Convergence plus rapide que Sigmoid
Pooling	Max Pooling	Robuste aux petits décalages
Device	CPU	(GPU non disponible lors de ce run)

2.6 Résultats et Analyse

2.6.1 Évolution des performances

Table 2.6 — Progression de la perte et de l’accuracy — LeNet sur CIFAR-10

Époque	Train Loss	Test Accuracy	Observation
1	1,6221	48,57 %	Démarrage rapide
2	1,3300	54,02 %	Forte progression
3	1,2065	56,95 %	—
4	1,1115	58,53 %	—
5	1,0376	61,32 %	Passage au-dessus de 60 %
6	0,9739	61,23 %	Léger plateau
7	0,9295	63,42 %	Meilleur point
8	0,8794	62,98 %	Résultat final

2.6.2 Performances par classe

Table 2.7 — Précision, rappel et F1 par classe (jeu de test)

Classe	Precision	Recall	F1-Score
avion	0,66	0,72	0,69
voiture	0,77	0,73	0,75
oiseau	0,47	0,59	0,52
chat	0,53	0,32	0,40
cerf	0,66	0,39	0,49
chien	0,51	0,60	0,55
grenouille	0,63	0,78	0,70
cheval	0,76	0,66	0,71
bateau	0,66	0,83	0,74
camion	0,71	0,66	0,69
Global	0,64	0,63	0,62

2.6.3 Visualisation des feature maps

Après la première couche convolutionnelle, on obtient 6 cartes de caractéristiques. Chacune correspond à un filtre différent et met en évidence un type de motif (bords, textures, zones colorées). C'est la preuve que le CNN apprend automatiquement des détecteurs de caractéristiques, sans qu'on les ait définis manuellement.

2.6.4 Comparaison CNN vs MLP

Un MLP simple ($3 \times 32 \times 32 \rightarrow 256 \rightarrow 128 \rightarrow 10$) a été entraîné sur les mêmes images :

Table 2.8 — Comparaison MLP vs LeNet (CNN) — CIFAR-10, 8 époques

Modèle	Paramètres	Accuracy (test)	Remarque
MLP simple	820 874	53,13 %	Beaucoup plus de paramètres, moins bon
LeNet (CNN)	83 126	62,98 %	$\approx 10\times$ moins de paramètres, meilleur

2.6.5 Analyse et discussion

- **Efficacité du partage de poids** : LeNet utilise environ 10 fois moins de paramètres que le MLP tout en étant nettement meilleur (62,98 % contre 53,13 %), grâce au partage des poids (le même filtre balaye toute l'image).
- **Invariance par translation** : grâce au pooling, le CNN reconnaît un motif même légèrement décalé.
- **Difficulté de CIFAR-10** : contrairement à des chiffres, ce sont des images naturelles couleur ; ≈ 63 % avec un petit LeNet en 8 époques est un résultat attendu. Les classes visuellement ambiguës souffrent le plus (chat : rappel 0,32 ; cerf : 0,39), tandis que bateau (0,83) et grenouille (0,78) sont mieux reconnues.
- **Pistes** : un réseau plus profond, de l'augmentation de données ou un entraînement plus long (et sur GPU) amélioreraient sensiblement la précision.

3. Partie III — RNN, LSTM, GRU et Seq2Seq

3.1 Présentation du problème

Cette partie porte sur la traduction automatique de l’anglais vers l’espagnol. La difficulté ne se limite pas à la traduction mot à mot : l’espagnol obéit à des règles grammaticales différentes de l’anglais.

- **Ordre des mots** : en anglais l’adjectif précède le nom (« the red car »), tandis qu’en espagnol il le suit (« el coche rojo »).
- **Accord en genre** : l’adjectif et l’article s’accordent avec le nom (« el coche rojo » au masculin, « la casa roja » au féminin).

Sur un corpus réel, s’ajoutent un vocabulaire de plusieurs milliers de mots, des constructions variées et de l’ambiguïté lexicale.

3.2 Corpus anglais–espagnol (réel — Tatoeba)

3.2.1 Source et nature des données

Le corpus provient du projet Tatoeba, distribution « Anki spa-eng » (manythings.org) — le même jeu de données que le tutoriel Seq2Seq officiel de PyTorch. Il contient de l’ordre de 140 000 paires traduites par des humains, au format anglais \t espagnol \t attribution.

3.2.2 Prétraitement

- **Normalisation** : minuscules, suppression des accents (ASCII), ponctuation isolée comme jetons.
- **Filtrage par longueur** : on ne conserve que les paires d’au plus 10 mots de chaque côté.
- **Filtrage par préfixes** : restriction à des débuts de phrases anglaises fréquents (i, you, he, she, we, they, it, this, that, tom, the).
- **Dédoublonnage, mélange et plafonnement** : plafond fixé à 12 000 paires ; le filtrage en produit 11 170, toutes conservées.
- **Mise en forme finale** : tokenisation, vocabulaires avec jetons spéciaux SOS/EOS/PAD, encodage en indices, padding à 12 jetons, mini-lots de 64.

Table 3.1 — Exemples de paires du corpus réel (après normalisation)

Anglais	Espagnol
i need a vacation .	necesito unas vacaciones .
it s tom s turn .	es el turno de tom .
i never get drunk .	nunca me emborracho .
we helped him .	lo ayudamos .

Table 3.2 — Statistiques du corpus

Propriété	Valeur
Paires brutes (Tatoeba / Anki)	$\approx 140\,000$
Filtrage appliqué	≤ 10 mots + préfixes fréquents
Paires retenues	11 170
Entraînement (85 %)	9 494 paires
Test (15 %)	1 676 paires
Vocabulaire anglais	2 805 mots (SOS/EOS/PAD inclus)
Vocabulaire espagnol	4 682 mots (SOS/EOS/PAD inclus)
Longueur maximale (MAX_LEN)	12 jetons (EOS inclus)

3.3 Modèle de Langage et Perplexité

Objectif probabiliste. Un modèle de langage estime la probabilité d’une séquence, factorisée par la règle de chaîne :

$$P(w_1, \dots, w_T) = \prod_t P(w_t \mid w_1, \dots, w_{t-1}).$$

L’entraînement maximise la vraisemblance, ce qui revient à minimiser l’entropie croisée moyenne.

Perplexité. Elle vaut $PPL = \exp(\text{perte})$ et s’interprète comme le facteur de branchement moyen (par exemple, une perte de 2,0 correspond à une perplexité de 7,39). Plus elle est basse, meilleure est la prédiction.

3.4 Architectures Récurrentes

Le même modèle de langage (embedding $\rightarrow$ cellule récurrente $\rightarrow$ couche linéaire) est instancié avec une cellule interchangeable.

3.4.1 RNN simple

À chaque pas, le RNN combine le mot courant et l’état caché précédent. Sa mémoire s’efface vite sur les longues séquences (gradient qui s’évanouit).

3.4.2 LSTM (Long Short-Term Memory)

Le LSTM ajoute une mémoire à long terme contrôlée par trois portes.

Table 3.3 — Les trois portes du LSTM et leur rôle

Porte	Rôle	Analogie
Porte d’oubli	Décide quelle information oublier	« Dois-je oublier le sujet actuel ? »
Porte d’entrée	Décide quelle information stocker	« Dois-je mémoriser ce nouveau sujet ? »
Porte de sortie	Décide quoi exposer en sortie	« Quelle information est utile maintenant ? »

3.4.3 GRU (Gated Recurrent Unit)

Version simplifiée du LSTM à deux portes, moins coûteuse pour des performances comparables sur des séquences courtes.

3.5 BPTT et Gradient Clipping

BPTT. On propage les erreurs à travers chaque pas de temps ; le gradient est un produit de jacobiens dont la norme se comporte comme $(\text{rayon spectral})^T$, d'où risque d'explosion ou d'évanouissement.

Gradient clipping. Lorsque la norme du gradient dépasse un seuil (ici 1,0), on la ramène à ce seuil, sans changer sa direction. Expérimentalement, sans clipping la perte du RNN oscille ; avec clipping, elle descend régulièrement.

3.6 Comparaison RNN / LSTM / GRU

Table 3.5 — Résultats comparatifs — modèle de langage (corpus réel)

Architecture	Perplexité finale	Temps (6 époques)	Remarque
RNN simple	7,97	99,4 s	Le plus rapide
LSTM	9,41	122,0 s	Le plus coûteux GRU
	7,67	114,4 s	Meilleure perplexité

Sur ces séquences courtes et avec seulement 6 époques, les trois cellules sont proches : le **GRU** obtient la meilleure perplexité (7,67), suivi du RNN (7,97), le LSTM étant ici légèrement en retrait (9,41) et le plus lent. L'avantage théorique des portes du LSTM/GRU se manifeste surtout sur de longues dépendances, peu présentes ici ; côté coût, le RNN reste le plus léger.

3.7 Système Seq2Seq Encodeur-Décodeur

L'encodeur (GRU) lit la phrase anglaise et la compresse en un vecteur de contexte de taille fixe ; le décodeur (GRU) génère la phrase espagnole mot par mot à partir de ce vecteur.

Table 3.6 — Hyperparamètres du système Seq2Seq

Hyperparamètre	Valeur	Rôle
Nombre d'époques	20	Convergence (perte finale $\approx 0,58$)
Taille de batch	64	Standard
Taille d'embedding	128	Représentation dense des mots
Taille d'état caché	256	Capacité mémoire du modèle
Teacher forcing ratio	0,5	50 % de supervision directe
Gradient clipping	1,0	Stabilité de l'entraînement
Longueur max (MAX_LEN)	12	Troncature / padding

3.7.1 Teacher Forcing

Avec une probabilité de 50 %, on fournit au décodeur le vrai mot cible (plutôt que sa prédiction) au pas suivant, ce qui stabilise et accélère l'apprentissage, au prix d'un léger écart entre entraînement et inférence (exposure bias).

3.8 Stratégies de Décodage

3.8.1 Décodage glouton

À chaque pas, on choisit le mot le plus probable : rapide mais myope.

3.8.2 Beam Search ($k = 3$)

On conserve les 3 meilleures hypothèses partielles à chaque pas et on retient la séquence au meilleur score : utile quand le modèle est incertain.

3.9 Évaluation : Score BLEU

Le BLEU mesure le recouvrement des n-grammes entre la traduction et la référence, avec une pénalité de brièveté. C'est la métrique standard en traduction automatique car plusieurs traductions correctes peuvent exister.

3.10 Résultats et Analyse

Table 3.7 — Performances du système Seq2Seq (corpus réel EN → ES)

Stratégie	BLEU moyen	Traductions exactes	Remarque
Greedy (glouton)	0,341	5,4 %	Rapide
Beam Search ($k = 3$)	0,346	—	Léger gain de BLEU

L'exactitude (traductions strictement identiques à la référence) n'a été mesurée que pour le dé-codage glouton.

Table 3.8 — Exemples de traductions (décodage glouton)

Anglais	Référence	Traduction obtenue
i lost everything .	lo perdi todo .	lo he perdido .
i ll do it myself .	lo hare yo mismo .	lo hare lo .
tom is punctual .	tom es puntual .	tom esta silbando .
i had to resign .	tuve que renunciar .	tuve que .

3.10.1 Analyse et discussion des résultats

- **Performance réaliste** : sur le corpus réel, le BLEU plafonne autour de **0,34** et seulement **5,4 %** des traductions sont exactes — très loin des ≈ 100 % d'un corpus jouet. Les exemples montrent des traductions plausibles mais souvent partielles ou erronées.
- **Goulot d'étranglement** : toute la phrase source est compressée dans un unique vecteur de contexte (256). Pour les phrases longues ou rares, cette compression perd de l'information — c'est la limite que le mécanisme d'attention vient lever.
- **Beam search vs greedy** : le beam search n'apporte qu'un gain marginal ($0,341 \rightarrow 0,346$), le modèle restant assez confiant à chaque pas.

- **Pistes d'amélioration** : plus d'époques et de données, capacité accrue, tokenisation sous-mots pour réduire les mots hors-vocabulaire, et surtout un mécanisme d'**attention** (voire un Transformer).

4. Synthèse et Comparaison des Trois Approches

Les trois parties de ce projet illustrent le principe fondamental du Deep Learning : **l'architecture doit refléter la structure des données**. Cette section explique le rôle de chaque approche, puis les compare selon plusieurs critères.

4.1 Explication des trois parties

4.1.1 Partie I — MLP et données tabulaires

Les données tabulaires (ici les 30 mesures cellulaires du cancer du sein) n'ont ni structure spa-tiale ni structure temporelle : chaque variable est une grandeur indépendante. Le **perceptron multicouche**, entièrement connecté, est donc l'architecture naturelle puisqu'il combine librement toutes les variables. Le jeu étant petit (569 exemples), la **régularisation** (Dropout, weight decay, early stopping) est déterminante pour éviter le surapprentissage. Le MLP atteint 94 % d'accuracy, au niveau d'une régression logistique : sur ce type de données, un réseau profond n'apporte qu'un gain marginal car la frontière de décision est quasi-linéaire.

4.1.2 Partie II — CNN et images

Une image possède une forte **structure spatiale** : les pixels voisins sont corrélés et un motif (bord, texture) peut apparaître à n'importe quelle position. Un MLP, en aplatissant l'image, ignore cette structure et exploserait en nombre de paramètres. Le **CNN** encode trois hypothèses adaptées : la localité (petits filtres), le **partage des poids** (le même filtre balaie toute l'image) et l'**invariance par translation** (pooling). Sur CIFAR-10, LeNet obtient 62,98 % d'accuracy avec environ 10 fois moins de paramètres qu'un MLP (53 %). Le biais inductif convolutionnel est ici décisif.

4.1.3 Partie III — RNN / Seq2Seq et séquences

Une phrase est une **séquence** de longueur variable où l'ordre des mots et les dépendances à distance comptent. Les cellules **récurrentes** entretiennent un état caché (mémoire) transmis d'un mot à l'autre ; les portes du **LSTM/GRU** atténuent la disparition du gradient. Pour la traduction, l'architecture **encodeur-décodeur** gère des séquences source et cible de longueurs et d'ordres différents. Sur le corpus réel Tatoeba, le BLEU plafonne à 0,34 : tout le sens de la phrase source est comprimé dans un unique vecteur de contexte (**goulot d'étranglement**), ce qui motive directement le mécanisme d'attention.

4.2 Comparaison transversale

Table 4.1 — Comparaison transversale des trois approches

Critère	MLP (Partie I)	CNN / LeNet (Partie II)	RNN–Seq2Seq (Partie III)
Type de données	Tabulaires	Images (RGB)	Séquences de texte
Dataset	Breast Cancer (569)	CIFAR-10 (60 000)	Tatoeba (11 170)
Principe / biais inductif	Connexions complètes entre toutes les variables	Localité, partage de poids, invariance par translation	Récurrance et mémoire (portes), encodeur–décodeur
Paramètres	2 114	83 126	≈ 2,7 M (embeddings + GRU + projection)
Régularisation	Dropout, weight decay, early stopping	Architecture (parcimonie) + données	Gradient clipping, teacher forcing
Métrique	Accuracy / F1	Accuracy	BLEU / perplexité
Performance	94,2 % (F1 0,94)	62,98 %	BLEU 0,34
Coût de calcul	Faible	Moyen	Élevé
Limite principale	N’exploite aucune structure	Capacité limitée (réseau peu profond)	Goulot d’étranglement du contexte

4.3 Lecture comparative

- **Adéquation architecture–données.** Chaque architecture encode des hypothèses sur les données : connexions complètes pour le tabulaire, convolutions pour les images, récurrance pour les séquences. Employer la mauvaise architecture gaspille des paramètres (un MLP sur images) ou ignore une information essentielle (l’ordre des mots).
- **Paramètres et efficacité.** Le CNN est le plus *parcimonieux au regard de sa tâche* : 10 fois moins de paramètres que le MLP équivalent sur images, pour une meilleure précision. Le MLP a peu de paramètres en absolu, mais seulement parce que l’entrée tabulaire est petite. Le Seq2Seq est de loin le plus lourd, à cause des deux tables d’embedding et de la projection sur un vocabulaire de plusieurs milliers de mots.
- **Difficulté des tâches et métriques.** Les performances ne sont pas directement comparables car les tâches et les métriques diffèrent : une classification binaire tabulaire (94 %) est plus facile qu’une classification d’images naturelles à 10 classes (63 %), elle-même plus simple qu’une traduction ouverte (BLEU 0,34, où une même phrase admet plusieurs traductions correctes).
- **Limites et perspectives.** Les trois limites pointent vers des extensions naturelles : exploiter des variables plus informatives pour le MLP, des réseaux plus profonds et de l’augmentation de données pour le CNN, et surtout le mécanisme d’**attention** (puis le **Transformer**) pour lever le goulot d’étranglement du Seq2Seq.

Conclusion Générale

Ce projet a permis d’implémenter, d’analyser et de comparer les trois grandes familles d’architectures du Deep Learning sur trois jeux de données réels.

Récapitulatif des performances

Table 4.2 — Performances obtenues sur les trois parties du projet

Partie	Modèle	Dataset	Performance
I	MLP (2 114 params)	Breast Cancer Wisc. (569)	Accuracy 94,2 %, F1 0,94
II	LeNet (83 126 params)	CIFAR-10 (60 000)	Accuracy 62,98 %
III	Seq2Seq GRU	Tatoeba EN → ES (11 170)	BLEU 0,34, exact 5,4 %

Leçons apprises

1. **L’architecture doit correspondre à la nature des données** : un MLP sur des données tabulaires, un CNN sur des images, des modèles récurrents sur des séquences.
2. **Le prétraitement est aussi important que le modèle** : normalisation (MLP), normalisation d’images et DataLoader (CNN), tokenisation et padding (Seq2Seq).
3. **La régularisation prévient le surapprentissage** : Dropout, gradient clipping et sélection du meilleur modèle sont des outils complémentaires.
4. **Comparer avec une baseline est essentiel** : la comparaison aux baselines classiques (Partie I : MLP vs régression logistique et forêt aléatoire) et entre MLP et CNN (Partie II) situe la valeur réelle de chaque architecture.
5. **Comprendre les outils en profondeur** : implémenter manuellement la convolution et le pooling, puis les valider contre PyTorch, éclaire le fonctionnement interne des CNN.

Perspectives d’amélioration

- **Mécanisme d’attention pour le Seq2Seq** : laisser le décodeur consulter toutes les positions de la source, éliminant le goulot d’étranglement.
- **Architecture Transformer** : le standard actuel en traitement du langage, fondé sur l’attention.
- **Réseaux plus profonds et augmentation de données** : pour améliorer nettement les performances sur CIFAR-10 (ResNet, data augmentation).
- **Transfer Learning** : réutiliser des modèles pré-entraînés (ResNet, BERT) pour d’excellents résultats même avec peu de données.

Projet réalisé dans le cadre du module Deep Learning — EMSI Casablanca, Filière Informatique 4 IAD, Année universitaire 2025–2026.